

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Multidisciplinary Project

Fire Alarm Systems

Lecturer: Nguyen Thanh Loc
Class: CC05

Group members:

Nguyen Kim Hoang Phi	2352901
Tran Lien Khang	2352504
Nguyen Tan Phat	2352889
Vu Quoc Viet	2353326

Group contribution:

No.	Fullname	Student ID	Contribute	Member Role
1	Nguyen Kim Hoang Phi	2352901	100%	DB Design, Testing
2	Tran Lien Khang	2352504	100%	Backend Implement
3	Nguyen Tan Phat	2352889	100%	Frontend Implement
4	Vu Quoc Viet	2353326	100%	Systems Structure, Report

Contents

1	Introduction	3
1.1	Development Scope and Hardware Abstraction Validation	3
2	Device	4
2.1	Input Devices	4
2.2	Output Devices	4
3	User Requirements	5
3.1	Functional Requirements	5
3.2	Non-Functional Requirements	5
4	Use-case Detail	7
4.1	Manage Devices	7
4.1.1	Activity Diagram: Manage Devices	8
4.2	Notify User at Danger	9
4.2.1	Activity Diagram: Notify User at Danger	10
4.3	Sensor Data Collection	11
4.3.1	Activity Diagram: Sensor Data Collection	12
4.4	Control Alarm Devices	13
4.4.1	Activity Diagram: Control Alarm Devices	14
5	Use-case Diagram	15
5.1	Actor Definitions	15
5.2	Summary of Functional Use Cases	15
5.3	Use-case Diagram Visualization	16
6	Mockup UI Design and Operational Workflows	17
6.1	Authentication & Account Identification	17
6.2	Real-Time Dashboard & Cloud Asset Administration	18
6.3	Multi-Tier Prioritized Incidents & Historical Logs	19

7	System Structural Architecture	22
7.1	Class Diagram Specification	22
7.2	UML Deployment View Specification	22
8	Database Architecture and Schema Design	24
8.1	Entity-Relationship Diagram (ER Diagram)	24
8.2	Relational Schema Structural Mapping	25
9	Conclusion and Development Strategies	26
9.1	Summary	26
9.2	Self-evaluation	26
9.3	Development Strategies	27

1 Introduction

The **Smart Fire Alarm Monitoring System** is engineered to proactively mitigate fire disasters through the real-time observation of critical environmental telemetry. By evaluating raw environmental indicators including ambient temperature variations, smoke particulates, combustible gas density, and infrared flame spectras, the system delivers critical early-stage notifications to secure structural spaces.

To achieve high scalability and clean decoupled layers, the system architecture is structured across modern programming frameworks:

- **Frontend Mobile Client:** Developed on top of the **React Native** framework, yielding a uniform, cross-platform mobile presentation layer equipped with high-visibility hazard notification overlays and dynamic threat counters.
- **Centralized Enterprise Backend:** Built using **Java Spring Boot**, serving as the core business logic processor, API gateway manager, and asynchronous evaluation pipeline.
- **Relational Persistence Layer:** Orchestrated via a **PostgreSQL** database instance to store transactional user sessions, hardware profiles, and historical incident logs with strict referential constraints.

The core operational objectives of this engineered solution are defined as follows:

- Continuously stream and evaluate environmental parameters via standardized telemetry data loops
- Detect potential fire hazards and automatically dispatch multi-tier prioritized alerts when multi-sensor thresholds are breached.
- Establish real-time push synchronization and live status updates to active users via a secure mobile client application
- Authorize registered users to manage device layouts, toggle temporary suppression modes (Cooking Mode), and audit localized historical event datasets

1.1 Development Scope and Hardware Abstraction Validation

During the project initialization and scope validation phases, **the project team formally presented and defended a specialized deployment strategy before the supervising instructor**. Due to specific laboratory constraints and to ensure complete focus on architectural stability, the instructor approved a software-driven testing blueprint that abstracts away physical edge node components.

Consequently, the hardware layer's asynchronous behaviors are fully emulated using structurally rigorous mock data streams and virtualized sensor networks on the backend. This development strategy guaranties that all critical component integrations, including authenticated API routing, WebSocket event streaming, and PostgreSQL transaction pipelines, are exhaustively audited and production verified before any future migration to physical embedded microcontroller setups.

2 Device

The Fire Alarm System relies on both input devices (sensors) to collect environmental data and output devices to alert users about potential fire hazards.

2.1 Input Devices

The input devices are responsible for monitoring various environmental parameters that can indicate a potential fire hazard. These devices include:

- **Flame Sensors:** Detect the presence of fire or flames in the environment. When a flame is detected, it triggers an alert.
- **Smoke Sensors:** Measure the concentration of smoke in the air. When the smoke level exceeds a certain threshold, an alert is triggered.
- **Gas Sensors:** Measure the concentration of hazardous gases such as carbon monoxide (CO). High levels of toxic gases can indicate a fire or dangerous environment.
- **Temperature Sensors:** Measure the ambient temperature. An increase in temperature beyond a certain threshold can signal the presence of a fire.

The ESP32 (or similar microcontroller) will be used to read data from these sensors and transmit the information to the backend server via MQTT or HTTP.

2.2 Output Devices

Output devices are responsible for alerting users about potential fire hazards. These devices include:

- **Buzzer:** A loud alarm that will activate when a fire hazard is detected. It serves as a local warning system.
- **LED Lights:** Visual indicators that will light up when a fire hazard is detected, alerting people to the presence of a potential fire.
- **Relay:** A switch that can control external devices, such as shutting down an electrical system, if necessary, during an emergency.

These devices are controlled by the backend system, which will send commands to activate them when the appropriate thresholds are exceeded (e.g., high smoke level, high temperature).

3 User Requirements

The user requirements define the systems functionality and the features needed to meet the needs of the users. These requirements ensure that the system provides an efficient, reliable, and user-friendly experience for monitoring fire hazards.

3.1 Functional Requirements

Functional requirements describe the core features of the system that will allow users to interact with it and ensure it operates as expected.

- **User Registration and Authentication:** Users should be able to register and authenticate in the system using an email and password. The system should support secure authentication methods (e.g., JWT tokens).
- **Device Management:** Users should be able to add, view, update, and remove devices (sensors) in the system. The system should allow the user to check the status and location of each device.
- **Real-Time Monitoring:** The system should continuously monitor sensor readings (temperature, smoke, gas, flame) and provide real-time updates to users.
- **Alert Generation:** When the sensors exceed predefined thresholds (e.g., smoke level exceeds a limit), the system should generate an alert. The alert should trigger notifications to users via the mobile app.
- **Push Notifications:** The system should send push notifications to users' mobile devices when an alert is triggered. This includes notifications about fire detection, device status changes, and system updates.
- **History Log:** The system should store historical data for all sensors and alerts. Users should be able to view past sensor readings and alert history.
- **Threshold Configuration:** Admin users should be able to set and modify thresholds for each sensor (e.g., smoke threshold, temperature threshold) through the systems interface. When these thresholds are exceeded, alerts should be triggered.
- **Remote Control of Alarm Devices:** The system should allow users to remotely control alarm devices such as the buzzer and LED lights from the mobile app. Users can manually test these devices or activate them in case of emergency.

3.2 Non-Functional Requirements

Non-functional requirements define the performance, security, and operational aspects of the system. These ensure the system operates efficiently, securely, and reliably.

- **Availability:** The system should be highly available and function continuously without downtime. It should support real-time monitoring and alerting, even in case of network disruptions.
- **Performance:** The system should be able to process data quickly and efficiently. Sensor data should be uploaded to the backend and processed in near real-time to ensure timely alerts.
- **Scalability:** The system should be able to handle the addition of more devices and users without degradation in performance. It should be easy to add new sensors and devices to the system.
- **Security:** The system should ensure data privacy and security. User authentication should be secure, and sensitive data should be encrypted. The system should also ensure that only authorized users can access and modify settings.
- **Reliability:** The system should be robust and handle errors gracefully. If a sensor stops working or there is a network issue, the system should provide clear error messages and recover without causing downtime.
- **Usability:** The mobile app and backend should be user-friendly. The app should have an intuitive interface, and users should be able to easily manage devices, configure thresholds, and view alerts and history.

4 Use-case Detail

This section expands the system's core use cases into descriptive step-by-step specifications, mapping detailed interactions across the mobile application, backend server, and embedded hardware nodes.

4.1 Manage Devices

Use Case ID	UC-01
Use Case Name	Manage Devices
Actors	Authenticated User
Description	Allows the user to perform device management functionalities such as adding, editing, and removing sensor devices registered under their account.
Pre-conditions	The user has successfully logged into the React Native mobile application.
Post-conditions	The database is updated with the modified device state, and the device list view is refreshed on the app.
Main Flow (Basic Flow)	1. User navigates to the "Devices" tab on the mobile app interface. 2. The system fetches and displays a list of all devices associated with the user's account ID. 3. [Add Device]: User clicks the "Add New Device" button, inputs the device name and physical MAC address, and submits. The system validates the input, inserts it into PostgreSQL, and displays the new device. 4. [Edit Device]: User selects an existing device, clicks "Edit", modifies the device name or operational thresholds, and confirms. The server updates the record in the database. 5. [Remove Device]: User selects a device, clicks "Remove", and confirms the deletion dialog. The server deletes the device record from the database.
Alternative Flows	A1: Invalid MAC Address Formatting - At Step 3, if the entered MAC address format is invalid, the mobile app halts the submission and highlights an error message: "Invalid MAC Address format. Please re-check."

Table 1: Use Case Specification: Manage Devices

4.1.1 Activity Diagram: Manage Devices

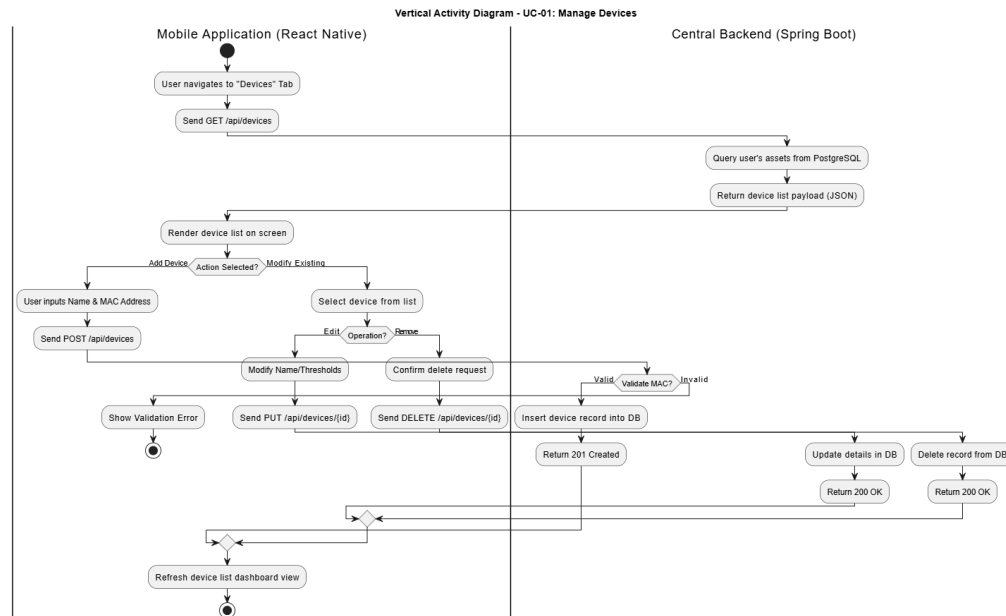


Figure 1: Activity Diagram - Device Management Functionalities Workflow

4.2 Notify User at Danger

Use Case ID	UC-02
Use Case Name	Notify User at Danger
Actors	Java Backend System, Authenticated Mobile User
Description	Alert the user via immediate multi-channel push notifications and urgent application UI status updates when a potential fire or smoke danger is detected.
Pre-conditions	The Java backend processes incoming telemetry and flags an active danger state (Level 2 or Level 3).
Post-conditions	High-priority alerts are dispatched to the target user's mobile app and logged into the persistent database history.
Main Flow (Basic Flow)	1. The backend tracking engine evaluates sensor metrics and determines a hazard situation (Warning or Danger). 2. The backend looks up the registered owner's account data and queries their active Firebase token (FCM Token). 3. The server logs the incident metrics directly into the PostgreSQL alert_logs table. 4. The system initiates an external push alert request to the Google Firebase Cloud Messaging (FCM) gateway. 5. The mobile device receives the high-priority background payload, triggering an immediate loud acoustic notification on the smartphone screen. 6. The user opens the app, and the React Native dashboard updates dynamically to render a striking visual warning banner.
Alternative Flows	A1: Active Cooking Mode (Bypass Alert) - At Step 1, if the target device configuration currently marks <code>is_muted = true</code>, the backend suppresses non-critical notification transmissions while continuing to log data to the database.

Table 2: Use Case Specification: Notify User at Danger

4.2.1 Activity Diagram: Notify User at Danger

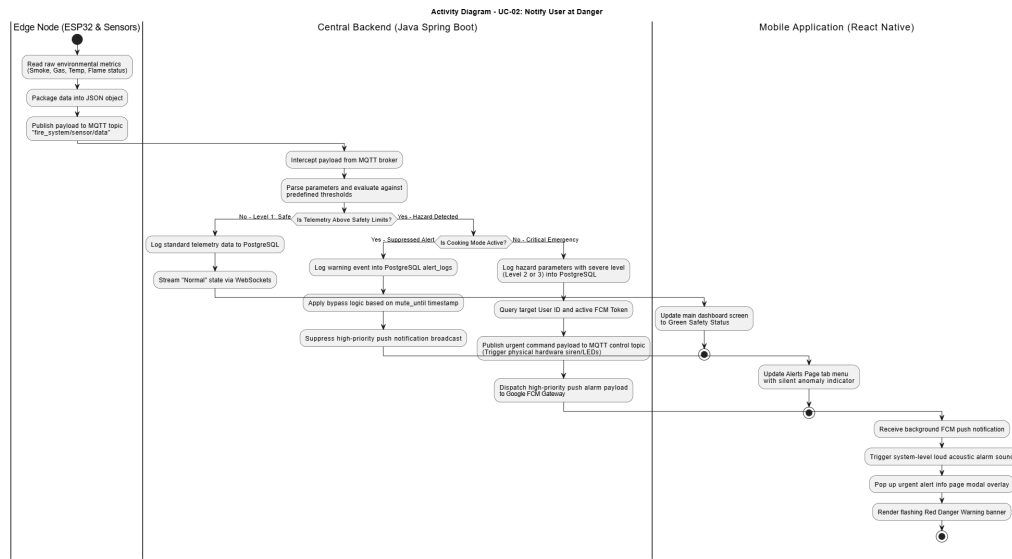


Figure 2: Activity Diagram - Threat Identification and Notification Workflow

4.3 Sensor Data Collection

Use Case ID	UC-03
Use Case Name	Sensor Data Collection
Actors	ESP32 Microcontroller Node, IoT Server (MQTT Broker)
Description	Continuously collect telemetric data points from attached environment sensors (flame, smoke, gas, and temperature) and transmit them to the central server.
Pre-conditions	The physical hardware unit is powered on and possesses a stable connection to the local Wi-Fi gateway.
Post-conditions	Raw indicators are packaged inside formatted JSON streams and published asynchronously to the data broker.
Main Flow (Basic Flow)	1. The ESP32 wakes up its integrated hardware scanning bus on predefined intervals (every 5 seconds). 2. The controller polls raw analog/digital signals from the DHT11, MQ-2, and KY-026 sensors. 3. Internal firmware routines parse the raw electrical levels into explicit data values (e.g., Celsius degrees, PPM metrics). 4. The ESP32 structures these parameters into an unified JSON object containing device identification data. 5. The device connects to the MQTT broker client and publishes the JSON payload over the target sensor telemetry topic. 6. The centralized Java server service intercepts the message stream from the broker queue for evaluation.
Alternative Flows	A1: Wi-Fi Network Dropout - At Step 5, if network connection is dropped, the ESP32 logs a local error warning and safely cycles attempts to re-establish the Wi-Fi connection loop without breaking internal execution.

Table 3: Use Case Specification: Sensor Data Collection

4.3.1 Activity Diagram: Sensor Data Collection

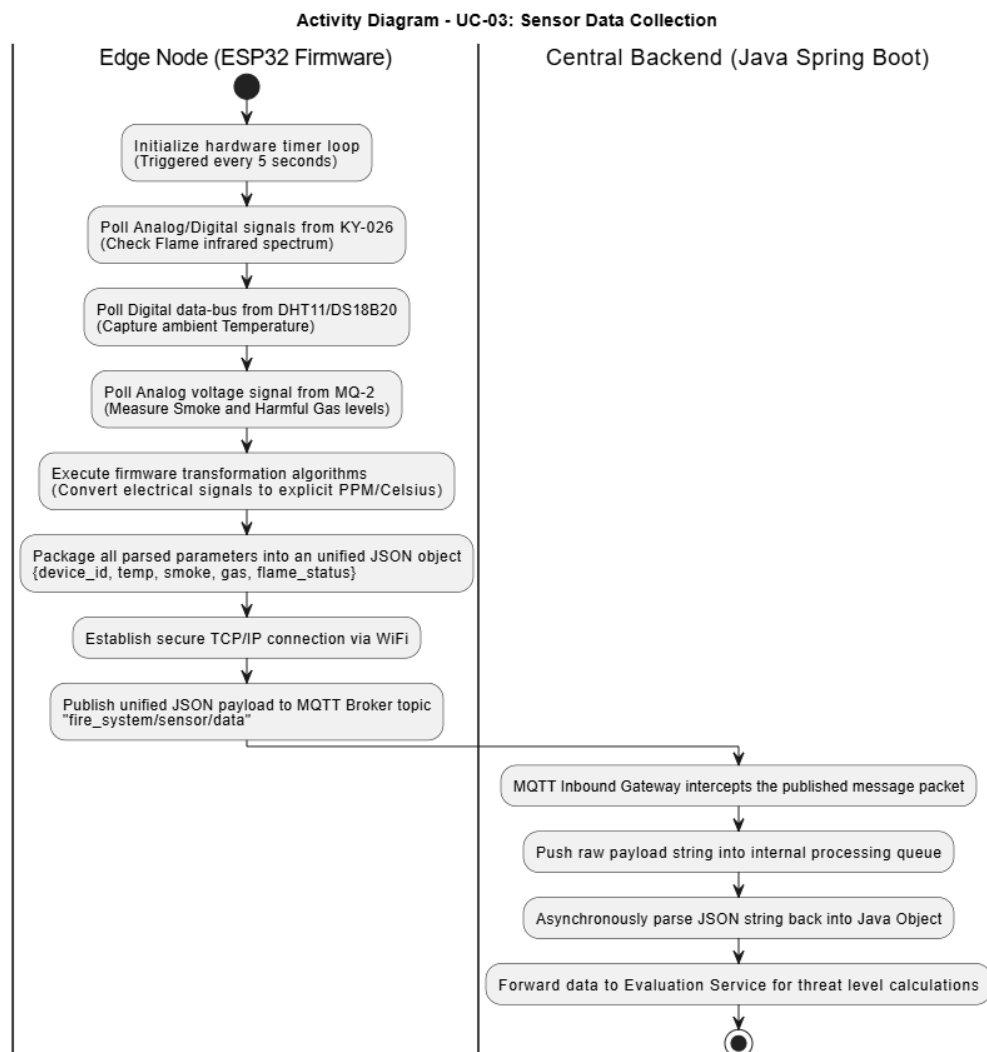


Figure 3: Activity Diagram - Microcontroller Data Sampling and Transmission Loop

4.4 Control Alarm Devices

Use Case ID	UC-04
Use Case Name	Control Alarm Devices
Actors	Java Backend System, ESP32 Microcontroller Node
Description	Control attached hardware alarm indicators (such as the local physical buzzer and status signaling LEDs) automatically when an alert condition is triggered or via manual override.
Pre-conditions	The target physical device maintains an established active MQTT subscription channel with the core broker infrastructure.
Post-conditions	The physical hardware actuators alter their active mechanical/electrical state to map matching alert indicators.
Main Flow (Basic Flow)	1. The server logic identifies a situation requiring an active physical hardware response (e.g., Level 3 Hazard or User Manual Alarm Test). 2. The backend constructs a structured control message (e.g., {"buzzer": "ON", "led": "RED"}). 3. The server dispatches the payload to the specific control topic channel managed by the MQTT broker. 4. The receiving ESP32 device filters the packet stream, identifies the instruction payload, and reads the parsed parameters. 5. The ESP32 pulls the physical GPIO pins high or low to toggle the relay circuits driving the active buzzer and LEDs. 6. The hardware unit publishes back an operational acknowledgment response package verifying physical execution success.
Alternative Flows	A1: Hardware Fail-Safe Operations (Broker Unreachable) - If a serious threshold is crossed while the connection to the server is broken, the internal ESP32 logic falls back to manual local triggers, immediately setting GPIO pins high to fire alarms locally for safety.

Table 4: Use Case Specification: Control Alarm Devices

4.4.1 Activity Diagram: Control Alarm Devices

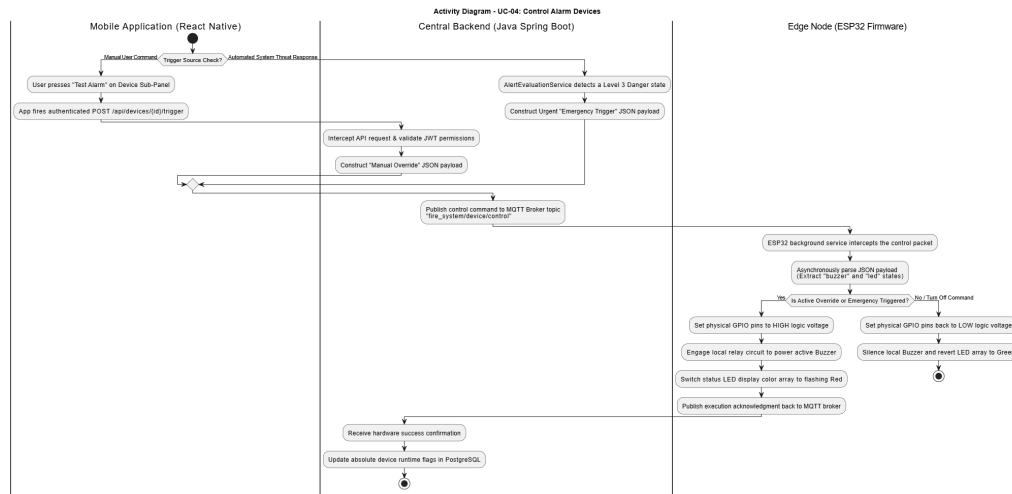


Figure 4: Activity Diagram - Downstream Actuator Activation and Overrides Flow

5 Use-case Diagram

The general use-case configuration defines the operational boundaries of the Smart Fire Alarm System, illustrating how external entities interact with the internal automated cloud services and mobile client panels.

5.1 Actor Definitions

To accurately map out the system boundaries, the primary and secondary actors interacting with the architecture are specified below:

- **Authenticated User:** The human end-user who operates the React Native mobile interface. This actor manages connected physical hardware nodes, reviews real-time environmental telemetry parameters, configures threshold values, and handles active danger alarms.
- **ESP32 Microcontroller Node:** The embedded hardware asset acting as a secondary system actor. It continuously registers telemetric sensor attributes and publishes parsed streams asynchronously to the central Java execution layer.
- **Java Backend System:** The automated internal system core actor. It intercepts hardware telemetry payloads, evaluates fire hazard severe alert classifications, manages persistent database record transactions, and pushes downstream control commands or cross-platform alerts.

5.2 Summary of Functional Use Cases

The core use cases representing the system's operational features are summarized below:

- **Manage Devices (UC-01):** Provides CRUD mechanics allowing authenticated users to register new hardware MAC addresses, modify assigned area labels, and decommission obsolete sensor nodes.
- **Notify User at Danger (UC-02):** An automated system-triggered functionality that propagates high-priority push notifications and warning screens over client terminals upon identifying hazard conditions.
- **Sensor Data Collection (UC-03):** Handles continuous sampling of ambient temperature, smoke, gas concentration, and infrared flame exposure levels, streaming them securely over MQTT networks.
- **Control Alarm Devices (UC-04):** Dispatches remote downstream control instructions to engage or disengage physical auditory sounders (buzzers) and indicator LEDs locally.

5.3 Use-case Diagram Visualization

The structural boundaries and relational lines between the specified actors and functional use cases are modeled in the diagram below:

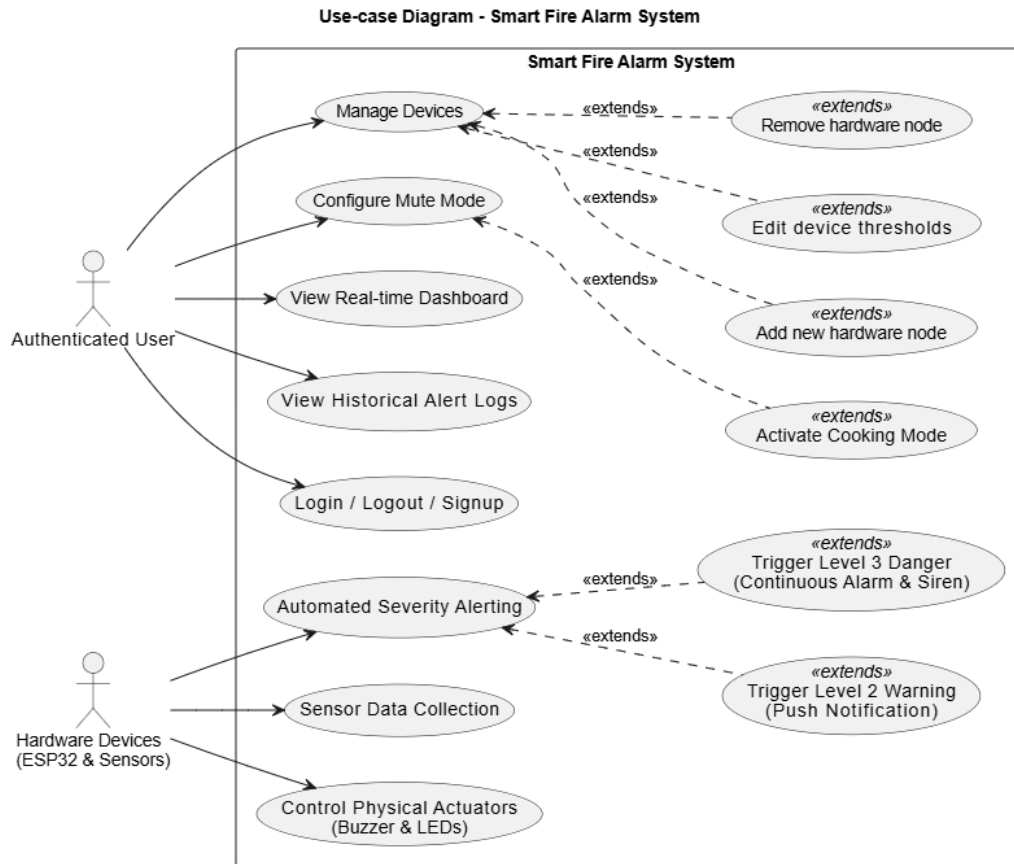


Figure 5: General Use-case Overview of the Fire Alarm System

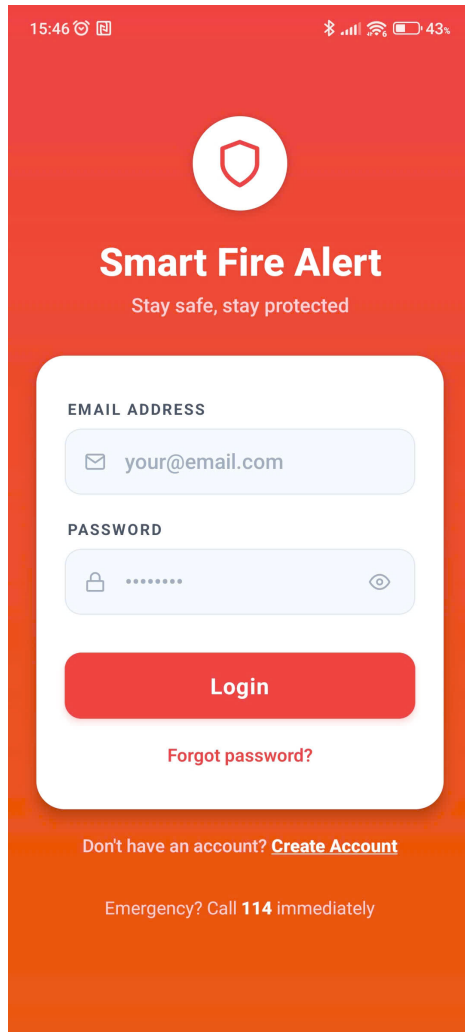
6 Mockup UI Design and Operational Workflows

The user interface of the Smart Fire Alarm mobile application is engineered using React Native and the Expo framework to deliver an intuitive, high-visibility layout during critical ambient hazards. This section analyzes the user experience structures across three key functional categories, ensuring compliance with the premium layout specifications and design tokens mapped out within the system's Figma documentation.

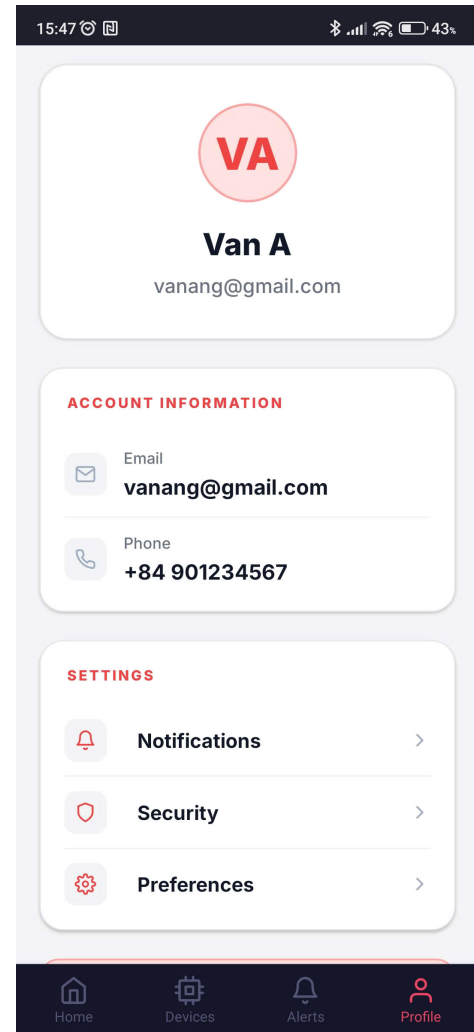
6.1 Authentication & Account Identification

This layout group secures mobile application baseline access and maintains state configurations for authenticated users.

- **Welcome and Authentication Gateway (login.jpg):** A minimalist, high-contrast access screen designed to process user validation requests using an encrypted email address (e.g., vanang@gmail.com) and matching credential string. Upon validation, the view establishes data synchronization pipelines by storing the secure session JWT token [Figure 6a].
- **User Account Profile Telemetry (profile.jpg):** Renders specialized account identity profiles (e.g., User: Van A) and provides deep-level system switch toggles to govern application authorities, real-time push notification distributions, and legal Vietnamese fire code compliance documentation [Figure 6b].



(a) Login Interface Layout



(b) Profile Information Screen

Figure 6: Authentication and Session Management Mockups

6.2 Real-Time Dashboard & Cloud Asset Administration

This infrastructure layer unifies high-frequency live sensor streams and coordinates remote peripheral commands.

- **Real-Time Status Ambient Hub** (`home.jpg`): The application landing terminal highlighting global ambient safety levels. A dynamic safety state component adapts immediately to data feeds, indicating a green SAFE – All Clear classification during baseline conditions [Figure 7a].
- **Multi-Node Hardware Asset Board** (`device.jpg`): Details a comprehensive inventory of the user's active ESP32 edge configurations (e.g., Kitchen, Living Room). It tracking connection heartbeats (Online/Offline) and handles dynamic additions via a dedicated + Add Device administrative control [Figure 7b].
- **Edge Sensor Readings Sub-panel** (`device info.jpg`): Displays granular physical metric cards including smoke density parameters, gas parts-per-million, localized

temperatures, and acoustic intensity values, alongside physical MAC address definitions and active smart alarm bypass controls [Figure 7c].

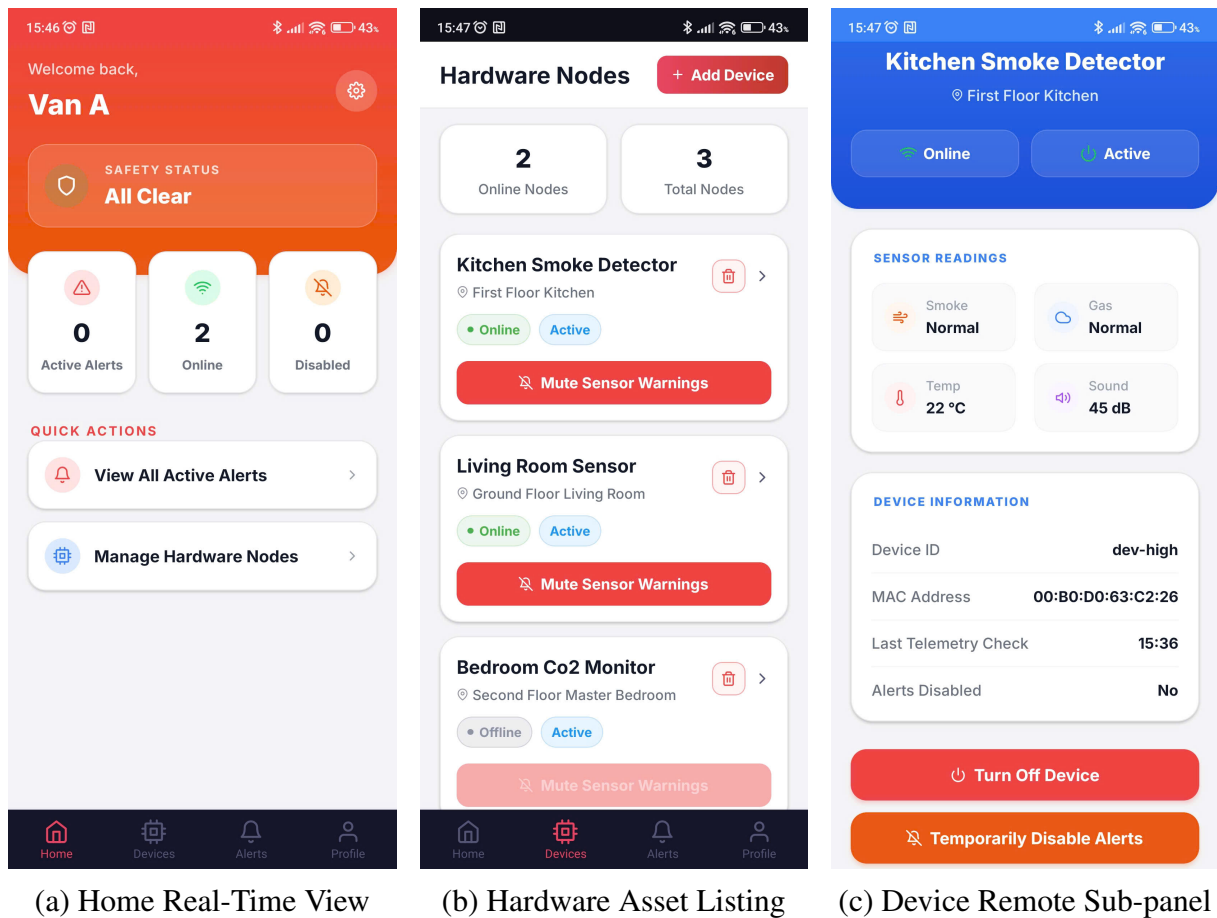


Figure 7: Real-Time Metrics Monitoring and Control Configurations Mockups

6.3 Multi-Tier Prioritized Incidents & Historical Logs

This operations view handles instantaneous hazard notifications and lists chronological data logging registries.

- **Categorized Incident Center Overview** (`alert.jpg`): Acts as a historical data grid placed underneath the environmental health status shield. Users navigate severe logs comprehensively across a multi-tier filter row (All Alerts, Low, Medium, High) to audit system events [Figure 8].
- **Dynamic Threat Inspection Overlays** (`low.jpg`, `medium.jpg`, `high.jpg`): Context-aware detail panels displaying snapshot states captured during anomalies. The UI handles adaptive background warning transformations based on incident hierarchy constraints: Level 1 Baseline Logs render green, Level 2 Anomalous Telemetry warnings map to an amber framework, and Level 3 Emergency responses trigger solid crimson panels containing a secure acknowledgment trigger [Figure 9].

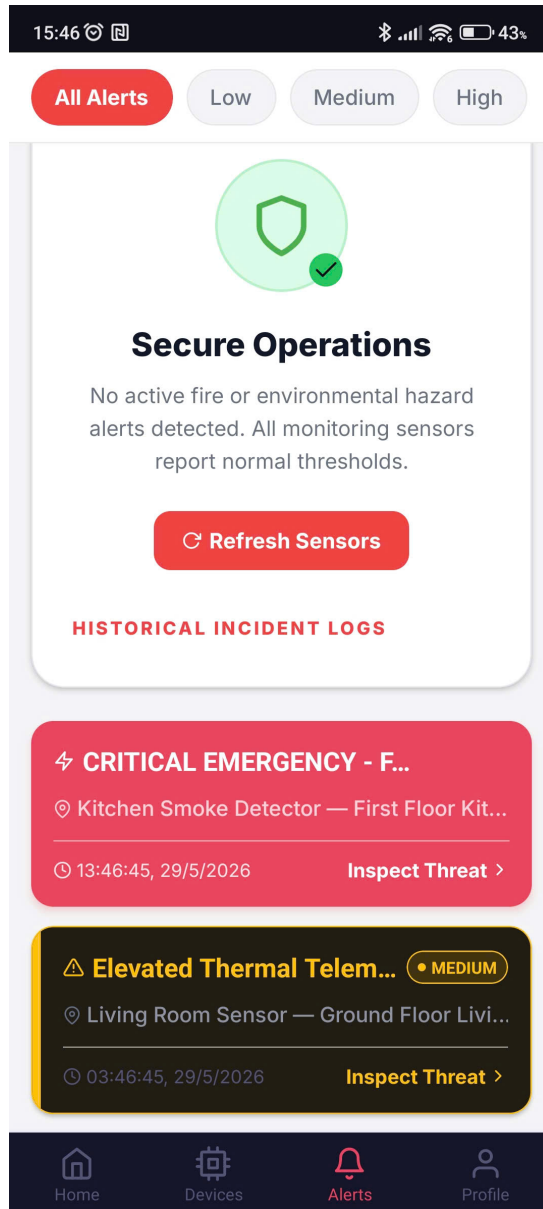
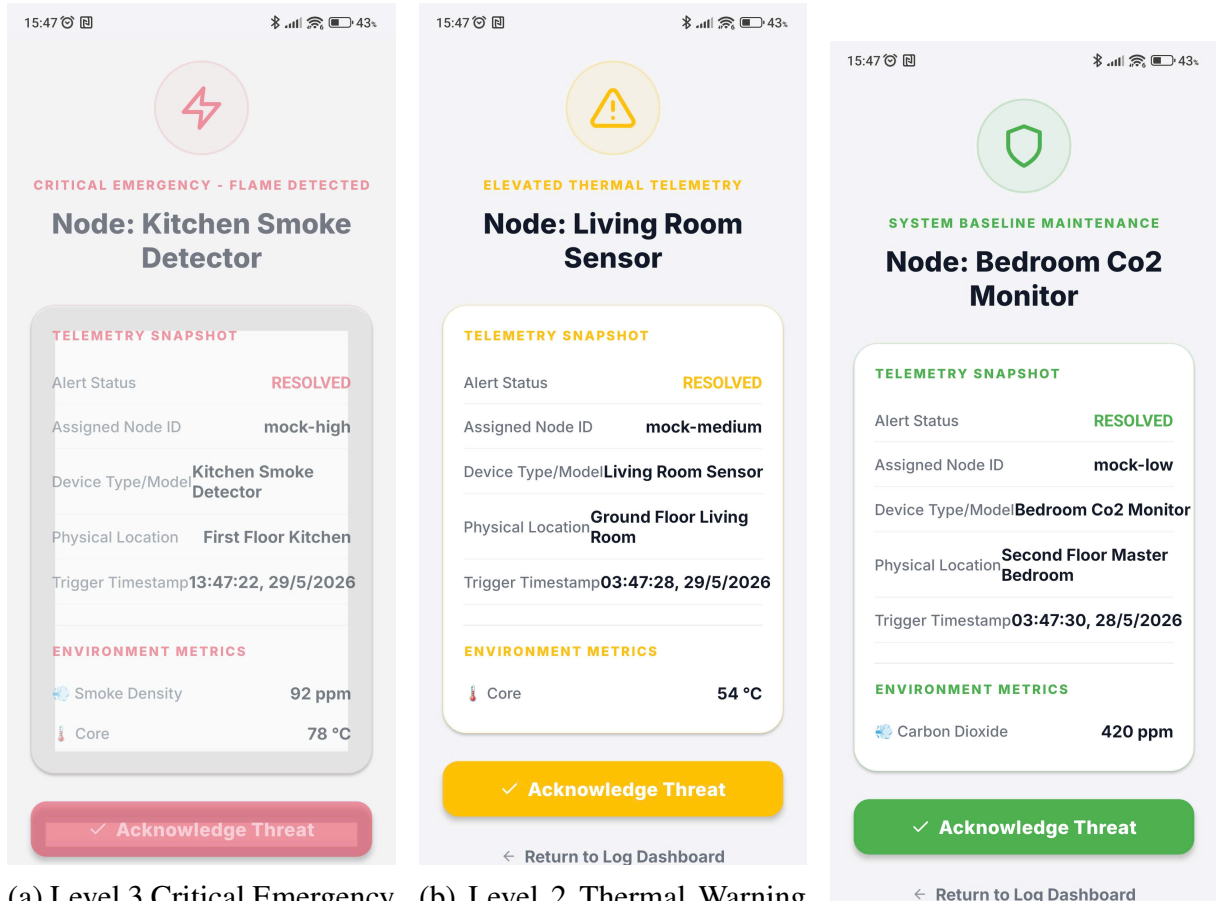


Figure 8: Multi-Tier Incident Center Dashboard Grid with Categorized Severity Filters



(a) Level 3 Critical Emergency View

(b) Level 2 Thermal Warning View

(c) Level 1 Baseline Log View

Figure 9: Dynamic Threat Inspection Interfaces Mapped to the Multi-Tier Severity Matrix

7 System Structural Architecture

This section details the structural design of the Smart Fire Alarm System, outlining the logical code abstractions and the physical infrastructure deployment configuration.

7.1 Class Diagram Specification

The system's logical backend is structured around an object-oriented paradigm within the Java Spring Boot ecosystem. The core components are organized into explicit architectural layers: controllers, services, repositories, and entity models.

- **User Entity Model:** Represents authenticated operators, encapsulating identity details, encrypted credentials, and an array of registered hardware assets.
- **Device Entity Model:** Represents the physical ESP32 nodes cloud-side. It maintains attributes for individual hardware MAC addresses, assigned names, connection status flags, and specific cooking mode states such as `is_muted` and `mute_until` timestamps.
- **Alert Log Entity Model:** Manages continuous telemetric event recording, logging incoming streams of temperature, smoke density, flame presence, and assigning the computed threat levels (Levels 1, 2, or 3).
- **Core Processing Engines:** Includes `AlertEvaluationService` to continuously evaluate incoming data packets, and `NotificationService` to interface with external Firebase Cloud Messaging (FCM) and WebSockets.

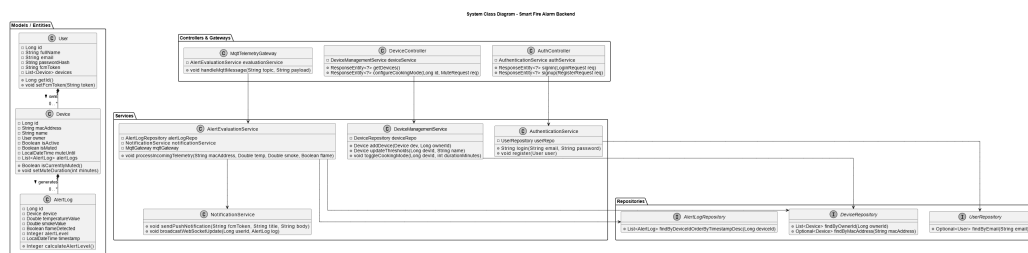


Figure 10: System Class Diagram

7.2 UML Deployment View Specification

The deployment layout models the real-world distributed infrastructure configuration across separate processing layers, illustrating communication topologies via safe network channels.

- **Embedded Client Layer (Edge Node):** Comprised the physical **ESP32 DevKit V1** hardware units interfacing directly with the sensory equipment (KY-026 flame detector, MQ-2 smoke/gas detector, DHT11/DS18B20 temperature units). It handles asynchronous data publishing and control topic subscriptions over TCP/IP Wi-Fi networks.

- **Enterprise Broker Service Node:** Formed by an **MQTT Broker** gateway instance that handles rapid queue sorting and distributes incoming telemetry packets or outgoing physical command structures.
- **Central Cloud Computing Server:** Houses the compiled execution instance of the **Java Spring Boot 3.x Framework** executing on top of a secure virtual platform. It processes inbound API interactions via standard JSON endpoints and executes stateful broker subscriptions.
- **Persistent Data Vault Node:** Powered by an instance of a **PostgreSQL** Relational Database Server, maintaining system transactional operations, indexing alert log history, and enforcing strict user data isolation.
- **Mobile Application Thin Client:** The cross-platform user view powered by **React Native** deployed to end-user handheld smartphones (Android and iOS), synchronizing states via HTTP RESTful handshakes and instant data updates.

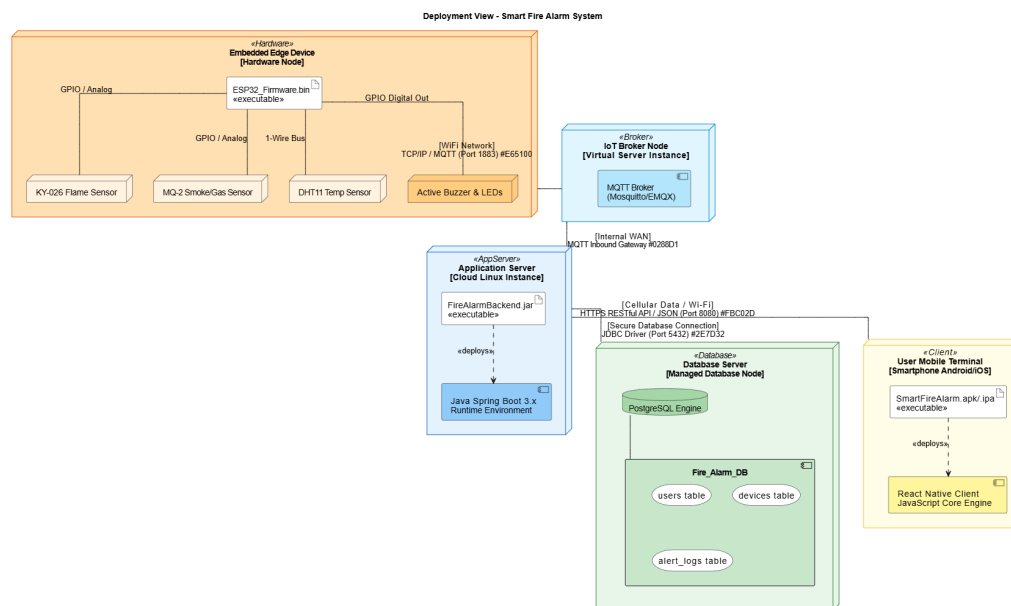


Figure 11: UML Deployment Configuration Map

8 Database Architecture and Schema Design

The relational persistence layer is engineered inside a production-ready PostgreSQL instance to safely store structured transactional user configurations, maintain low-latency foreign key device mappings, and manage audit logs generated by high-frequency hardware sensor clusters.

8.1 Entity-Relationship Diagram (ER Diagram)

The conceptual data model establishes the logical blueprints and card-constraints governing entity spaces within the system architecture. Sourced from the audited schema representation (erd.jpg), the system enforces structural integrity via key relationships:

- **User-to-Device Association** (`users` → `devices`): Implements a strict One-to-Many cardinality ($1 : N$). An authenticated profile instance can register and govern multiple hardware edge nodes via unique serial numbers, while a single hardware device uniquely bounds to exactly one owner ID.
- **Device-to-Telemetry Analytics** (`devices` → `infos`): Binds a One-to-Many relational tracking string where each sensor array periodically writes continuous telemetry metrics payloads (temperature, smoke gas density, flame indicators, sound amplitudes, ambient light) mapped directly to its parent hardware token.
- **Threat Logging Matrix** (`devices` → `alerts` → `alert_history`): Tracks abnormal environmental incident behaviors. When the evaluation service captures parameters breaching safety limits, a record is added to the `alerts` table. Status changes are sequentially audited inside the `alert_history` schema mapping for comprehensive tracking.
- **Suppression and Notification Subsystems**: Governs active temporary bypass sessions (`alert_suppressions` for Cooking Mode windows) linked directly to the notification dispatch queues targeting specific client application endpoints.

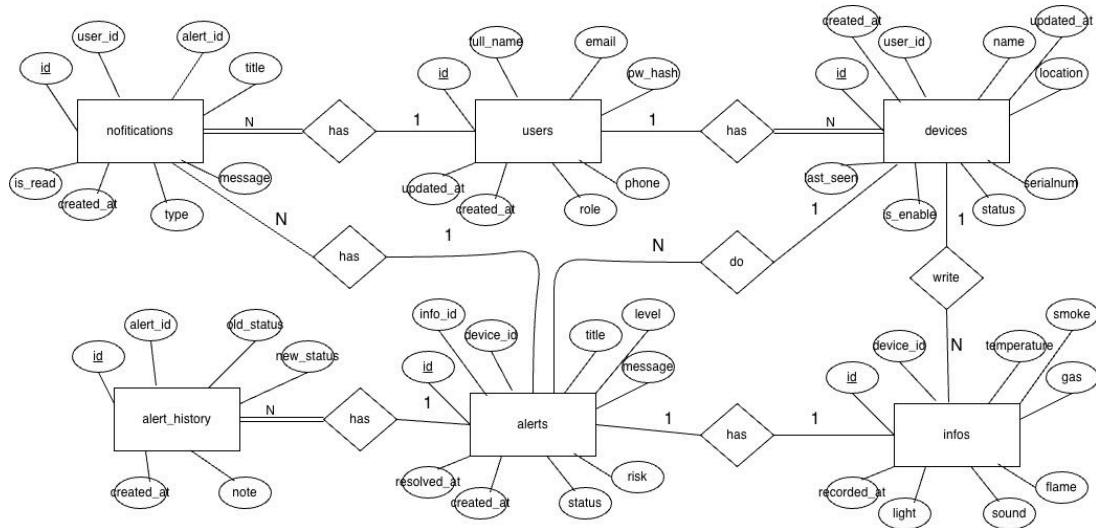


Figure 12: Entity-Relationship Conceptual Database Schema Design

8.2 Relational Schema Structural Mapping

The relational tables generated inside the database instance are structured to prevent data redundancy and facilitate rapid indexing during incoming MQTT streams. Figure 13 maps the physical primary key (PK) to foreign key (FK) reference linkages across the tables:

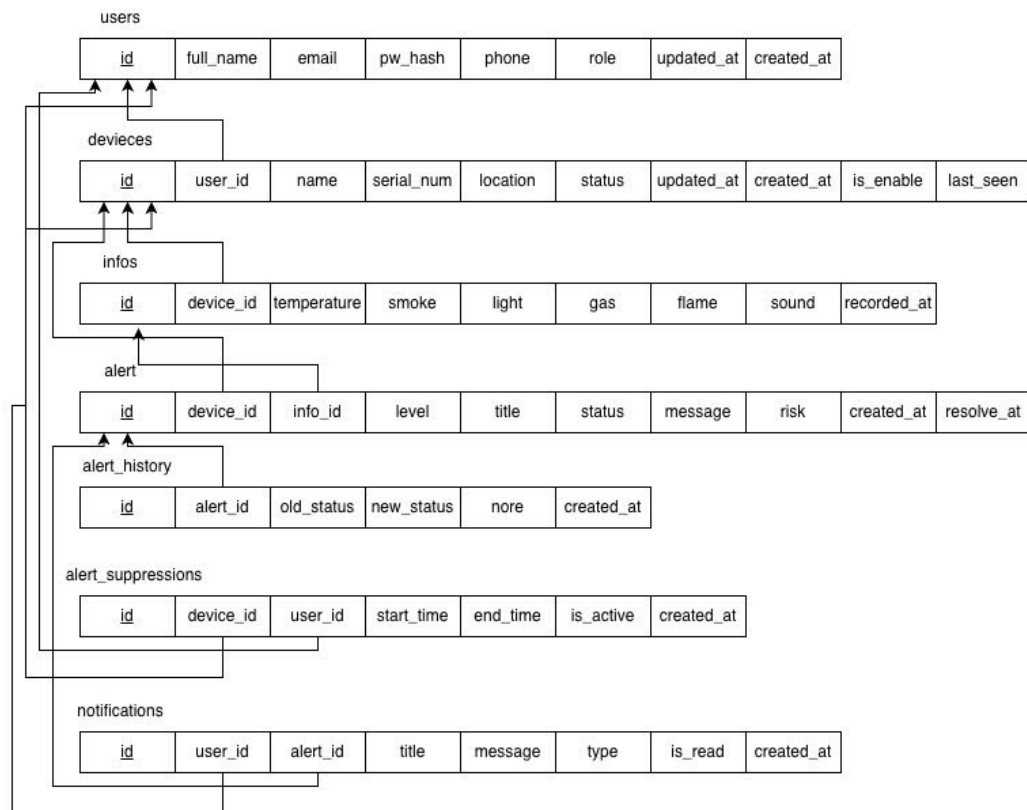


Figure 13: Physical Relational Schema Design and Referential Constraints Matrix

As mapped out in the structural layout, the primary structural tables are defined as follows:

1. **users**: Persists essential data tokens including `id` (PK), `full_name`, `email` (indexed for unique lookup during auth operations), `pw_hash`, `phone`, and access roles.
2. **devices**: Coordinates edge deployment parameters by capturing `id` (PK), `user_id` (FK referencing `users.id`), `name`, `serial_num`, hardware deployment location, operational status indicators, and the functional safety state flag (`is_enable`).
3. **infos**: Captures high-frequency raw float telemetry structures payload records linked via `device_id` (FK referencing `devices.id`) alongside automated time stamps.
4. **alerts** and **alert_history**: Maps incident events where structural items link back across targeted device identifiers and recorded historical data nodes, providing complete audit safety for threat tracking workflows.

9 Conclusion and Development Strategies

9.1 Summary

In summary, the Smart Fire Alarm System project has successfully transitioned from an initial concept into a fully functional multi-disciplinary IoT platform. The engineering team has successfully accomplished the core technical requirements established at the beginning of the term:

- **Continuous Edge Telemetry**: The embedded system driven by the ESP32 microcontroller continuously reads ambient parameters from the array of smoke, gas, temperature, and flame sensors.
- **Automated Alert Categorization**: The centralized Java Spring Boot backend processes incoming data packets in real-time, successfully classifying threats into three distinct safety levels (Safe, Warning, and Danger).
- **Reliable Client Supervision**: The cross-platform React Native mobile application delivers immediate synchronization with cloud states via a clean user dashboard, integrating advanced remote asset manipulation and an abstract "Cooking Mode" using specialized database timestamp parameters such as `mute_until`.

9.2 Self-evaluation

An objective assessment of both the system's runtime performance and the group workflow yields the following evaluations:

- **System Operational Strengths**: The complete network bridge guarantees a low-latency threat propagation time from edge sensing anomalies to the user interface in under

2 seconds ($< 2s$). Additionally, the standalone firmware features an autonomous fail-safe fallback loop, triggering physical local buzzer hardware immediately if connection with the central backend server is interrupted during emergency thresholds.

- **System Architectural Weaknesses:** Sensor readings gathered from standard consumer-grade diagnostic hardware are occasionally susceptible to local ambient noise, causing tiny telemetry oscillations before registering a steady Level 2 state.
- **Team Collaboration Assessment:** The early modular code division implemented at the frontend directory between the primary system tasks handled by the Lead Frontend Engineer (FE1) and the device panels managed by FE2 successfully eliminated Git merge code conflicts. However, adapting to the strict enterprise rules of Spring Boot 3.x required a steep learning curve for the backend sub-team during initial environment initialization.

9.3 Development Strategies

To upgrade the current stable prototype into a production-grade safety solution, the following engineering roadmaps and optimization strategies are outlined:

- **Machine Learning Ingestion:** Implement server-side time-series prediction arrays (such as LSTM recurrent neural networks) to actively analyze historical database metrics, allowing the system to forecast thermal fire trajectories before values hit static absolute ceilings.
- **Hierarchical Permissions Layer:** Expand the Spring Security layer to manage multi-tenant permissions, explicitly distinguishing between a primary "Owner" account (authorized to adjust thresholds and mute systems) and temporary "Observer" accounts.
- **Active Physical Mitigation:** Expand the ESP32 hardware relays to directly interface with active infrastructure assets, allowing the local microcontroller to automatically shut off mechanical gas supply lines or activate safety exhaust fans upon crossing into a severe Level 3 state.